

2. 两数相加

两数相加

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Medium (45.25%)	11204	0	-	-	0

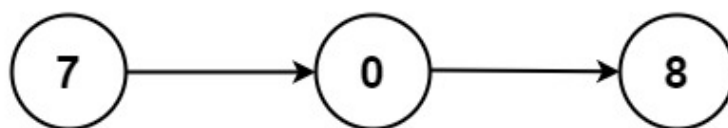
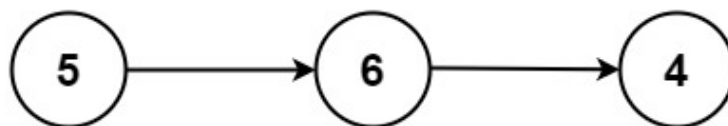
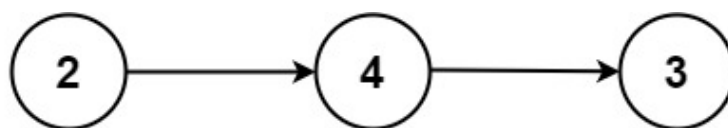
- Tags
- Companies

给你两个 非空 的链表，表示两个非负的整数。它们每位数字都是按照 逆序 的方式存储的，并且每个节点只能存储 一位 数字。

请你将两个数相加，并以相同形式返回一个表示和的链表。

你可以假设除了数字 0 之外，这两个数都不会以 0 开头。

示例 1:



输入: 11 = [2,4,3], 12 = [5,6,4]

输出: [7,0,8]

解释: 342 + 465 = 807.

示例 2:

输入: 11 = [0], 12 = [0]

输出: [0]

示例 3:

输入: 11 = [9,9,9,9,9,9,9], 12 = [9,9,9,9]

输出: [8,9,9,9,9,0,0,0,1]

提示:

- 每个链表中的节点数在范围 [1, 100] 内
- $0 \leq \text{Node.val} \leq 9$
- 题目数据保证列表表示的数字不含前导零

Discussion | Solution

2. 方法

LeetCode 2: 两数相加 (详细解析)

1. 问题理解

目标: 将两个非负整数相加, 这两个整数由链表表示, 其中每个节点包含一个数字, 并且数字是逆序存储的。

输入: 两个非空链表的头节点 11 和 12。

输出: 一个新的链表, 表示两个输入链表所代表数字的和, 同样是逆序存储。

示例:

- 输入: 11 = [2,4,3], 12 = [5,6,4]
- 输出: [7,0,8]
- 解释: $342 + 465 = 807$

关键点:

- 数字是逆序存储的, 这意味着链表的头节点代表个位, 第二个节点代表十位, 以此类推。
- 这使得模拟小学加法 (从个位开始加, 处理进位) 变得非常方便。

2. 基本思路

我们可以模拟手动加法的过程:

1. 从两个链表的头节点 (个位) 开始。
2. 将对应位置的数字相加, 并加上来自上一位的进位 (carry)。
3. 计算当前位的数字 (和 % 10) 和新的进位 (和 / 10)。
4. 创建一个新节点存储当前位的数字, 并将其添加到结果链表中。

5. 移动到两个链表的下一个节点，并重复此过程，直到两个链表都遍历完毕且没有进位。

3. 迭代法实现

步骤

1. 创建一个哑节点 `dummy` 作为结果链表的伪头节点，方便插入。
2. 创建一个 `current` 指针，用于构建结果链表，初始指向 `dummy`。
3. 初始化进位 `carry` 为 0。
4. 使用指针 `p1` 和 `p2` 分别遍历 `l1` 和 `l2`。
5. 循环进行，只要 `p1`、`p2` 任意一个不为空，或者还有进位 `carry`，就继续：
 - a. 获取 `p1` 和 `p2` 当前节点的值。如果某个指针为空，则其值为 0。
 - b. 计算当前位的和： $\text{sum} = \text{val1} + \text{val2} + \text{carry}$ 。
 - c. 更新进位： $\text{carry} = \text{sum} / 10$ 。
 - d. 计算当前位要放入新节点的数字： $\text{digit} = \text{sum} \% 10$ 。
 - e. 创建新节点 `new ListNode(digit)`，并将其连接到 `current.next`。
 - f. 移动 `current` 指针到新建的节点：`current = current.next`。
 - g. 移动 `p1` 和 `p2` 指针到下一个节点（如果它们不为空）。
6. 循环结束后，返回 `dummy.next`。

代码实现

Users\cc\lcpr\2.add-two-numbers.java

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    /**
     * 将两个逆序存储数字的链表相加
     * @param l1 第一个链表
     * @param l2 第二个链表
     * @return 表示和的新链表
     */
    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        // 创建哑节点，简化头节点处理
        ListNode dummy = new ListNode(0);
        // current 指针用于构建结果链表
```

```

ListNode current = dummy;
// 初始化进位为 0
int carry = 0;

// 使用 p1 和 p2 遍历 l1 和 l2
ListNode p1 = l1;
ListNode p2 = l2;

// 循环条件：只要任意一个链表还有节点，或者还有进位，就继续
while (p1 != null || p2 != null || carry != 0) {
    // 获取当前节点的值，如果节点为空则视为 0
    int val1 = (p1 != null) ? p1.val : 0;
    int val2 = (p2 != null) ? p2.val : 0;

    // 计算当前位的和（包括进位）
    int sum = val1 + val2 + carry;

    // 更新进位（为下一位做准备）
    carry = sum / 10;
    // 计算当前位的值
    int digit = sum % 10;

    // 创建新节点，存储当前位的值
    current.next = new ListNode(digit);
    // 移动 current 指针到新节点
    current = current.next;

    // 移动 p1 和 p2 指针到下一个节点（如果存在）
    if (p1 != null) {
        p1 = p1.next;
    }
    if (p2 != null) {
        p2 = p2.next;
    }
}

// 返回哑节点的下一个节点，即结果链表的头节点
return dummy.next;
}
}

```

4. 代码解析

哑节点 (Dummy Node):

- `ListNode dummy = new ListNode(0);` 创建一个值为 0 的哑节点。

- 作用：简化链表头部的插入操作。我们不需要特殊处理第一个节点的插入，直接将所有新节点都添加到 `current.next` 即可。最后返回 `dummy.next` 就是真正的头节点。

Current 指针：

- `ListNode current = dummy`；初始化 `current` 指向哑节点。
- 作用：始终指向结果链表的最后一个节点，方便在其后添加新节点。

Carry 变量：

- `int carry = 0`；初始化进位为 0。
- 作用：存储上一位计算产生的进位，用于当前位的计算。

循环条件：

- `while (p1 != null || p2 != null || carry != 0)`
- 这个条件确保了所有位都被处理，包括：
 - 两个链表都还有节点。
 - 一个链表比另一个长，需要继续处理较长链表的剩余部分。
 - 两个链表都结束了，但最高位还有一个进位需要处理（例如 $9+1=10$ ）。

获取节点值：

- `int val1 = (p1 != null) ? p1.val : 0`;
- `int val2 = (p2 != null) ? p2.val : 0`;
- 安全地获取当前节点的值，如果链表已经遍历完（指针为 `null`），则认为其值为 0。

计算和与进位：

- `int sum = val1 + val2 + carry`；计算总和。
- `carry = sum / 10`；计算新的进位（整数除法）。
- `int digit = sum % 10`；计算当前位应存储的数字（取模）。

构建新链表：

- `current.next = new ListNode(digit)`；创建新节点并连接到结果链表的末尾。
- `current = current.next`；移动 `current` 指针到新的末尾节点。

移动输入链表指针：

- `if (p1 != null) p1 = p1.next`;
- `if (p2 != null) p2 = p2.next`;
- 将 `p1` 和 `p2` 移到下一个节点，为下一轮计算做准备。

5. 复杂度分析

时间复杂度： $O(\max(m, n))$

- 其中 `m` 和 `n` 分别是两个链表的长度。
- 循环的次数取决于较长链表的长度以及最后可能产生的进位。

空间复杂度: $O(\max(m, n))$

- 需要创建一个新的链表来存储结果。
- 新链表的长度最多是较长链表的长度加一（如果最后有进位）。

6. 如何在面试中讲解

1. 理解问题和关键点:
 - “这道题要求我们将两个用链表表示的非负整数相加。关键在于数字是逆序存储的，这正好方便我们模拟从个位开始的加法运算。”
2. 提出核心思路:
 - “我们可以模拟小学竖式加法。从两个链表的头节点（个位）开始，逐位相加，同时处理进位。”
 - “我们需要一个变量来记录进位 `carry`。”
3. 讲解实现步骤（迭代法）:
 - “为了方便构建结果链表，我们可以使用一个哑节点 `dummy` 和一个 `current` 指针。”
 - “我们用 `p1` 和 `p2` 遍历两个输入链表，同时检查 `carry` 是否为 0。”
 - “在循环中，我们取出 `p1` 和 `p2` 的值（如果为空则为 0），加上 `carry`，得到 `sum`。”
 - “根据 `sum` 计算出当前位的值 (`sum % 10`) 和新的进位 (`sum / 10`)。”
 - “创建一个新节点存储当前位的值，并将其添加到 `current` 后面，然后移动 `current`。”
 - “最后移动 `p1` 和 `p2`。”
 - “循环结束后，返回 `dummy.next`。”
4. 强调边界处理:
 - “循环条件 `p1 != null || p2 != null || carry != 0` 很重要，它能处理链表长度不一以及最后有进位的情况。”
 - “在取值时，要判断指针是否为空，为空则视为 0。”
5. 分析复杂度:
 - “时间复杂度是 $O(\max(m, n))$ ，因为我们需要遍历完两个链表。”
 - “空间复杂度也是 $O(\max(m, n))$ ，因为我们需要创建新的链表来存储结果。”

这个解法清晰、直接，并且有效地模拟了加法过程。

找到具有 1 个许可证类型的类似代码